TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 14: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 18 questions requiring written explanations or short proofs about approximation ratios and local search. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

A1. Define what it means for a polynomial-time algorithm A to be a *ρ-approximation algorithm*, separately for minimization and maximization problems. Why is $\rho = 1$ the best possible value, and what does a larger ρ mean?

A2. Every approximation proof this week follows the same pattern: instead of comparing the algorithm's value $A(I)$ directly to $\mathrm{OPT}(I)$, we compare it to a *computable bound* $B(I)$. Explain why we cannot simply compare to $\mathrm{OPT}(I)$ directly, and state the two chained inequalities the proof must establish (for a minimization problem).

A3. Distinguish a *constant-factor* approximation, a *PTAS*, and an *FPTAS*. Order them from weakest to strongest guarantee, and explain the difference between a PTAS and an FPTAS in terms of how the running time depends on ε.

A4. State the Minimum Vertex Cover problem. Then describe the maximal-matching algorithm and prove that the set it returns is in fact a vertex cover.

A5. For the maximal-matching Vertex Cover algorithm, prove the lower bound $\mathrm{OPT} \geq |M|$, where M is the matching the algorithm builds. Why is it essential that the edges of M are pairwise vertex-disjoint?

A6. Combine the previous two facts into the full 2-approximation proof for Vertex Cover. Explain in one sentence why adding *both* endpoints of each matched edge (rather than just one) is what makes the analysis work.

A7. State the Minimum Set Cover problem and the greedy algorithm. Then prove that an element e first covered when greedy selects a set covering t uncovered elements is "charged" exactly $1/t$, and that the greedy cover size equals $\sum_e \mathrm{cost}(e)$.

A8. Complete the analysis of greedy Set Cover: prove that the elements of any single optimal set S_j receive total charge at most $H_{|S_j|} \leq H_n$, and conclude that greedy is an $H_n = O(\log n)$ approximation. (State clearly where you use the greedy choice rule.)

A9. State the Load Balancing (makespan) problem. Prove the two lower bounds on the optimal makespan: $\mathrm{OPT} \geq \frac{1}{m}\sum_j t_j$ and $\mathrm{OPT} \geq \max_j t_j$.

A10. Prove that greedy list scheduling is a 2-approximation for load balancing. Identify exactly where each of the two lower bounds from Q9 is used.

A11. State (without full proof) the improved guarantee for Longest-Processing-Time (LPT) scheduling. Explain intuitively why sorting jobs in *decreasing* order helps, and why the "last/critical job" on the bottleneck machine is small when there are at least two jobs on it.

A12. State the k-center problem and the greedy farthest-point algorithm. Why is the *metric* (triangle-inequality) assumption necessary – what goes wrong without it?

A13. Prove that greedy farthest-point selection is a 2-approximation for k-center. Your proof should produce $k+1$ sites that are pairwise far apart and apply pigeonhole over the k optimal centers.

A14. State the Metric TSP problem and the MST-based algorithm. Prove the lower bound $w(T) \leq \mathrm{OPT}$ relating the MST weight to the optimal tour.

A15. Complete the MST-based Metric TSP proof: explain the doubling/Eulerian-circuit and shortcutting steps, and show why shortcutting cannot increase the tour length. State the final 2-approximation bound. How does Christofides improve the constant?

A16. State the 0/1 Knapsack problem and the FPTAS via rounding-and-scaling. With scaling factor $\mu = \varepsilon\, p_{\max}/n$, prove the quality guarantee $\sum_{i\in S} p_i \geq (1-\varepsilon)\mathrm{OPT}$, and explain why the scaled DP runs in time polynomial in n and $1/\varepsilon$.

A17. Describe the local-search framework: feasible solution, neighbourhood, improving move, local optimum. For Max-Cut with the single-vertex-flip neighbourhood, prove that every local optimum cuts at least $m/2$ edges, and conclude it is a 2-approximation.

A18. Hill climbing can get stuck at a local optimum that is far from global. Explain how *random restarts* and *simulated annealing* each address this. For simulated annealing, state the acceptance probability for a worsening move and describe the role of the temperature schedule.

Part B: Quiz

B1. (Multiple choice) For a *minimization* problem, a 2-approximation algorithm guarantees that its output value is:

(a) at most $\frac{1}{2}$ of the optimum

(b) at most 2 times the optimum

(c) at least 2 times the optimum

(d) exactly the optimum

B2. (True/False) The Vertex Cover algorithm that takes both endpoints of a maximal matching is a 2-approximation, even though it never computes the optimal cover.

B3. (Short answer) In the maximal-matching Vertex Cover algorithm, if the matching M has 7 edges, what is the size of the cover returned, and what is the best lower bound this gives on OPT?

B4. (Multiple choice) The greedy Set Cover algorithm achieves an approximation ratio of:

(a) 2

(b) $\frac{3}{2}$

(c) $H_n = O(\log n)$

(d) $1 + \varepsilon$

B5. (True/False) Greedy list scheduling for load balancing assigns each job to the machine of currently *smallest* load.

B6. (Short answer) For $m = 3$ machines and total processing time $\sum_j t_j = 30$ with largest job $\max_j t_j = 7$, what is the best lower bound on the optimal makespan you can state from the two standard bounds?

B7. (Multiple choice) The greedy farthest-point k-center algorithm relies on which property of the distance function for its 2-approximation guarantee?

(a) symmetry only

(b) the triangle inequality

(c) that all distances are integers

(d) nothing – it works for arbitrary distances

B8. (True/False) For Metric TSP, the weight of a minimum spanning tree is a lower bound on the length of the optimal tour.

B9. (Short answer) An FPTAS for Knapsack returns a solution of value at least $(1 - \varepsilon)\text{OPT}$. If $\varepsilon = 0.1$ and $\text{OPT} = 200$, what is the worst-case value the FPTAS may return?

B10. (True/False) At a single-vertex-flip local optimum of Max-Cut, every vertex has at least half of its incident edges crossing the cut, which forces the cut to contain at least $m/2$ edges.

Part C: Solutions

Part A

A1. A is a **ρ-approximation** (with $\rho \geq 1$) if for every instance I: for *minimization*, $A(I) \leq \rho \cdot \mathrm{OPT}(I)$; for *maximization*, $A(I) \geq \frac{1}{\rho}\mathrm{OPT}(I)$. The value $\rho = 1$ means the algorithm is exactly optimal on every input (the best possible, since by definition $A(I)$ can be no better than $\mathrm{OPT}(I)$). A larger ρ means a weaker guarantee – the solution may be up to ρ times worse (minimization) or only a $1/\rho$ fraction as good (maximization) as optimal.

A2. We cannot compare to $\mathrm{OPT}(I)$ directly because computing $\mathrm{OPT}(I)$ is the NP-hard problem we are trying to avoid – there is no efficient way to know its value. Instead we find a quantity $B(I)$ we *can* reason about and that bounds the optimum, then prove two chained inequalities: (1) $B(I) \leq \mathrm{OPT}(I)$ (the bound is valid) and (2) $A(I) \leq \rho \cdot B(I)$ (the algorithm is close to the bound). Chaining gives $A(I) \leq \rho \cdot B(I) \leq \rho \cdot \mathrm{OPT}(I)$, all without computing $\mathrm{OPT}(I)$.

A3. A **constant-factor** approximation guarantees $\rho = c$ for a fixed constant independent of the input (e.g. $\rho = 2$). A **PTAS** (polynomial-time approximation scheme) achieves $\rho = 1 + \varepsilon$ for any fixed $\varepsilon > 0$, in time polynomial in n but possibly exponential in $1/\varepsilon$ (e.g. $n^{O(1/\varepsilon)}$). An **FPTAS** achieves $1 + \varepsilon$ in time polynomial in *both* n *and* $1/\varepsilon$ (e.g. $O(n^3/\varepsilon)$). Weakest to strongest: constant-factor $<$ PTAS $<$ FPTAS. The distinction between PTAS and FPTAS is precisely the dependence on ε: an FPTAS stays efficient even as $\varepsilon \to 0$, while a PTAS may blow up.

A4. **Minimum Vertex Cover:** given $G = (V, E)$, find a smallest $S \subseteq V$ such that every edge has at least one endpoint in S. **Algorithm:** scan the edges; whenever an edge (u, v) has neither endpoint yet in S, add *both* u and v to S (and record (u, v) in a matching M). S **is a cover:** consider any edge (x, y). If it were uncovered ($x, y \notin S$), then when the scan reached (x, y) neither endpoint was in S, so the algorithm would have added them – contradiction. Hence every edge has an endpoint in S.

A5. The edges of M are pairwise vertex-disjoint: once (u, v) is added, both u and v are in S, so any later edge sharing u or v fails the "neither endpoint in S" test and is not added to M. Now *any* vertex cover C must cover each edge of M, i.e. contain at least one endpoint of each M-edge. Because the M-edges share no endpoints, these chosen endpoints are all *distinct*, so $|C| \geq |M|$. Applying this to an optimal cover gives $\mathrm{OPT} \geq |M|$. Disjointness is essential: if two M-edges shared an endpoint, a single vertex could cover both, and we could not conclude $|C| \geq |M|$.

A6. The algorithm returns $S =$ endpoints of M, so $|S| = 2|M|$ (each edge contributes two distinct vertices). By Q5, $\mathrm{OPT} \geq |M|$. Therefore $|S| = 2|M| \leq 2\,\mathrm{OPT}$, a 2-approximation. Adding both endpoints (rather than one) is what guarantees S is a cover *and* that $|S| = 2|M|$ ties cleanly to the lower bound $|M|$; choosing one endpoint per matched edge would not guarantee coverage of all edges.

A7. **Minimum Set Cover:** given universe U ($|U| = n$) and subsets $S_1, \ldots, S_m$ with $\bigcup S_i = U$, find the fewest sets whose union is U. **Greedy:** while uncovered elements remain, pick the set covering the most currently-uncovered elements; add it; repeat. **Charging:** when greedy picks a set covering t previously-uncovered elements, we assign cost $1/t$ to each of those t elements – the single set used is "paid for" by its newly-covered elements, total $t \cdot (1/t) = 1$.

Each element is charged exactly once, at the moment it is first covered. Summing, $\sum_e \text{cost}(e)$ telescopes to the total number of sets greedy picks.

A8. Fix an optimal set S_j and order its elements $e_1, \ldots, e_{|S_j|}$ by the time greedy first covers them. Just before greedy covers e_k, the elements $e_k, \ldots, e_{|S_j|}$ are all still uncovered, so S_j *itself* could cover at least $|S_j|-k+1$ uncovered elements. *Here we use the greedy rule:* greedy picks the set covering the *most* uncovered elements, so the set it actually picks covers at least $|S_j| - k + 1$, giving each newly-covered element (including e_k) cost $\leq \frac{1}{|S_j|-k+1}$. Summing, $\sum_{e \in S_j} \text{cost}(e) \leq \sum_{k=1}^{|S_j|} \frac{1}{|S_j|-k+1} = H_{|S_j|} \leq H_n$. Finally, let $\mathcal{O}$ be an optimal cover; every element lies in some $S_j \in \mathcal{O}$, so $|\text{greedy}| = \sum_e \text{cost}(e) \leq \sum_{S_j \in \mathcal{O}} \sum_{e \in S_j} \text{cost}(e) \leq |\mathcal{O}| \cdot H_n = \text{OPT} \cdot H_n$.

A9. **Load Balancing:** m identical machines, jobs with times $t_1, \ldots, t_n$; assign each job to a machine; load L_k = sum of times on machine k; minimize makespan $\max_k L_k$. **Bound 1 (average):** the total work $\sum_j t_j$ is distributed over m machines, so by averaging some machine has load $\geq \frac{1}{m}\sum_j t_j$, hence the makespan is $\geq \frac{1}{m}\sum_j t_j$; this holds for every assignment, including the optimal one, so $\text{OPT} \geq \frac{1}{m}\sum_j t_j$. **Bound 2 (max job):** the machine running the largest job has load $\geq \max_j t_j$, so $\text{OPT} \geq \max_j t_j$.

A10. Let machine k achieve the makespan L, and let j be the last job placed on it. Just before placing j, machine k had load $L - t_j$, and since greedy always places on a least-loaded machine, *every* machine then had load $\geq L - t_j$. Summing over m machines, the work placed so far is $\geq m(L - t_j)$, so $L - t_j \leq \frac{1}{m}\sum_i t_i \leq \text{OPT}$ (*using Bound 1*). Also $t_j \leq \max_i t_i \leq \text{OPT}$ (*using Bound 2*). Adding, $L = (L - t_j) + t_j \leq 2\,\text{OPT}$.

A11. LPT (sort jobs in decreasing order, then greedy) achieves makespan $\leq \frac{4}{3}\text{OPT}$ (more precisely $(\frac{4}{3} - \frac{1}{3m})\text{OPT}$). Intuition: large jobs are the hard part; scheduling them first, while many small jobs remain to "fill the gaps," lets the small jobs balance the machines afterward. If the critical (last) job on the bottleneck machine is *not alone* on that machine, then – because jobs are placed in decreasing order and at least two jobs are on the (most-loaded) machine – there are at least $m+1$ jobs of size $\geq t_j$, which forces $t_j \leq \frac{1}{3}\text{OPT}$ (otherwise the total work exceeds what m machines can hold within OPT). Then $L = (L - t_j) + t_j \leq \text{OPT} + \frac{1}{3}\text{OPT} = \frac{4}{3}\text{OPT}$. The remaining case (critical job alone) is handled directly and turns out optimal.

A12. **k-center:** given sites P in a metric space and integer k, choose k centers C minimizing $r(C) = \max_p \min_{c \in C} d(p, c)$. **Greedy:** start with any center; repeatedly add the site *farthest* from the current center set, until k centers are chosen. The triangle inequality is necessary because the proof bounds the distance between two sites served by the same optimal center via $d(a, b) \leq d(a, c^*) + d(c^*, b)$. Without it, two sites both close to a common center could still be arbitrarily far from each other, so "farthest point" selection carries no guarantee, and indeed no constant-factor approximation is possible for general (non-metric) distances.

A13. Let $r = r(C)$ be the radius returned, realized by some site p at distance r from its nearest center. Consider the k chosen centers together with p: that is $k + 1$ sites. By the greedy rule, each center was chosen as the point farthest from the previously chosen centers, and the leftover distance of p is r; consequently every pair among these $k + 1$ sites is at distance $\geq r$. Suppose for contradiction $r > 2r^*$. There are only k optimal centers, so by pigeonhole two of the $k + 1$ sites, say a and b, share the same optimal center c^*, giving $d(a, c^*), d(b, c^*) \leq r^*$. By the triangle inequality $d(a, b) \leq 2r^* < r$, contradicting $d(a, b) \geq r$. Hence $r \leq 2r^*$: a 2-approximation.

A14. Metric TSP: n cities with symmetric distances satisfying the triangle inequality; find a minimum-length tour visiting each city once and returning to start. **Algorithm:** build an MST T, then output the preorder (DFS) traversal of T as the visiting order. **Lower bound:** take an optimal tour and delete any one edge. What remains is a Hamiltonian *path*, which is a spanning tree; its weight is $\leq$ the tour length. Since the MST is the lightest spanning tree, $w(T) \leq$ (this path) $\leq$ OPT.

A15. Double every edge of T to get $2T$; now all degrees are even, so $2T$ has an Eulerian circuit of total length exactly $2w(T)$. Traverse it and *shortcut*: when about to revisit a city, jump directly to the next unvisited one – this yields the preorder order and a valid tour. Each shortcut replaces a path between two cities by the direct edge, which by the triangle inequality is no longer; so the tour length $\leq 2w(T)$. Combined with $w(T) \leq \mathrm{OPT}$, the tour $\leq 2w(T) \leq 2\mathrm{OPT}$, a 2-approximation. **Christofides** improves the constant to $\frac{3}{2}$ by adding a minimum-weight perfect matching on the *odd-degree* vertices of T (instead of doubling all edges), producing an Eulerian graph of weight $\leq \mathrm{OPT} + \frac{1}{2}\mathrm{OPT}$.

A16. 0/1 Knapsack: items with profits p_i, weights w_i, capacity W; pick a subset of weight $\leq W$ maximizing total profit. **FPTAS:** set $\mu = \varepsilon\, p_{\max}/n$, round profits down to $\hat{p}_i = \lfloor p_i/\mu \rfloor$, and solve exactly under $\hat{p}_i$ by the profit-indexed DP. **Quality:** rounding gives $\mu\hat{p}_i \leq p_i \leq \mu\hat{p}_i + \mu$. Let O be optimal under true profits and S the DP's set (optimal under $\hat{p}$), so $\sum_{i\in S}\hat{p}_i \geq \sum_{i\in O}\hat{p}_i$. Then $\sum_{i\in S} p_i \geq \mu\sum_{i\in S}\hat{p}_i \geq \mu\sum_{i\in O}\hat{p}_i \geq \sum_{i\in O} p_i - n\mu = \mathrm{OPT} - \varepsilon p_{\max} \geq (1-\varepsilon)\mathrm{OPT}$, using $\mathrm{OPT} \geq p_{\max}$. **Running time:** each $\hat{p}_i \leq p_{\max}/\mu = n/\varepsilon$, so total rounded profit $\leq n^2/\varepsilon$, and the DP runs in $O(n \cdot \sum_i \hat{p}_i) = O(n^3/\varepsilon)$ – polynomial in n and $1/\varepsilon$.

A17. Framework: start from any feasible solution; the *neighbourhood* is the set of solutions reachable by a small "move"; repeatedly move to a strictly better neighbour (*improving move*); stop at a *local optimum*, where no neighbour is better. **Max-Cut (flip neighbourhood):** for each vertex v, let $d_{\mathrm{cut}}(v)$ and $d_{\mathrm{same}}(v)$ be its numbers of cut / same-side edges (summing to $\deg v$). Flipping v changes the cut by $d_{\mathrm{same}}(v) - d_{\mathrm{cut}}(v)$. At a local optimum flipping never helps, so $d_{\mathrm{same}}(v) \leq d_{\mathrm{cut}}(v)$, i.e. $d_{\mathrm{cut}}(v) \geq \frac{1}{2}\deg v$ for all v. Summing, $2 \cdot (\mathrm{cut}) = \sum_v d_{\mathrm{cut}}(v) \geq \frac{1}{2}\sum_v \deg v = \frac{1}{2}(2m) = m$, so the cut $\geq m/2$. Since $\mathrm{OPT} \leq m$, this is a 2-approximation.

A18. Random restarts: run hill climbing from many random initial solutions and keep the best local optimum found; since different starts lie in different "basins of attraction," more restarts raise the probability of reaching the global optimum. **Simulated annealing:** occasionally accept a *worsening* move to escape local optima, with acceptance probability $e^{-\Delta/T}$ for a move that worsens the objective by $\Delta > 0$, where T is a temperature. The schedule starts T high (most moves accepted, broad exploration) and gradually lowers it toward 0 (only improving moves accepted, settling into a good solution); a sufficiently slow schedule converges to the global optimum in the limit.

Part B

B1. (b) at most 2 times the optimum. For minimization, a ρ-approximation guarantees $A(I) \leq \rho \cdot \mathrm{OPT}(I)$.

B2. True. It returns the endpoints of a maximal matching M, of size $2|M| \leq 2\,\mathrm{OPT}$, never computing the optimal cover – yet the matching is a valid lower bound.

B3. Cover size $= 2 \times 7 = \mathbf{14}$ vertices; lower bound OPT $\geq |M| = \mathbf{7}$.

B4. **(c)** $H_n = O(\log n)$**.** This is the greedy Set Cover guarantee, and (up to constants) the best possible unless P = NP.

B5. **True.** Greedy list scheduling assigns each job in turn to a machine of currently minimum load.

B6. Bound 1: $\frac{1}{m}\sum_j t_j = 30/3 = 10$. Bound 2: $\max_j t_j = 7$. The best (largest) lower bound is $\max(10, 7) = \mathbf{10}$.

B7. **(b) the triangle inequality.** The 2-approximation proof uses $d(a, b) \leq d(a, c^*) + d(c^*, b)$.

B8. **True.** Deleting an edge from any tour leaves a spanning tree of weight $\leq$ the tour, so the minimum spanning tree weight $\leq$ OPT.

B9. $(1 - 0.1) \times 200 = 0.9 \times 200 = \mathbf{180}$.

B10. **True.** This is exactly the local-optimum condition $d_{\text{cut}}(v) \geq \frac{1}{2}\deg v$ for every vertex, which sums to a cut of size $\geq m/2$.